

Lecture – 8

Advanced Sorting Methods –Part 4

M.G.Noel A.S Fernando (PhD)

Professor

NSBM/Dept. of Information Systems Engineering, UCSC



Counting Sort Algorithm

- **Counting Sort** is a **non-comparison-based** sorting algorithm.
- It is particularly efficient when the range of input values is small compared to the number of elements to be sorted.
- The basic idea behind **Counting Sort** is to count the **frequency** of each distinct element in the input array and use that information to place the elements in their correct sorted positions.
- For example, for input [1, 4, 3, 2, 2, 1], the output should be [1, 1, 2, 2, 3, 4]
- The count is stored in an auxiliary array and the sorting is done by mapping the count as an index of the auxiliary array.

Working of Counting Sort

Step 1: Find out the maximum element (let it be max) from the given array.

	0	1	2	3	4	5	6	7	Max
arr[]=	2	5	3	0	2	3	0	3	5

Step 2: Initialize a cntArr[] of length max+1 with all elements as 0. This array will be used for sorting the occurrences of the elements of the input array.

	0	1	2	3	4	5
cntArr[]=	0	0	0	0	0	0

Working of Counting Sort cont...

- **Step 3:** In the `cntArr[]`, store the count of each unique elements of the input array at the respective indices

	0	1	2	3	4	5
cntArr[]=	2	0	2	3	0	1

	0	1	2	3	4	5	6	7
arr[]=	2	5	3	0	2	3	0	3

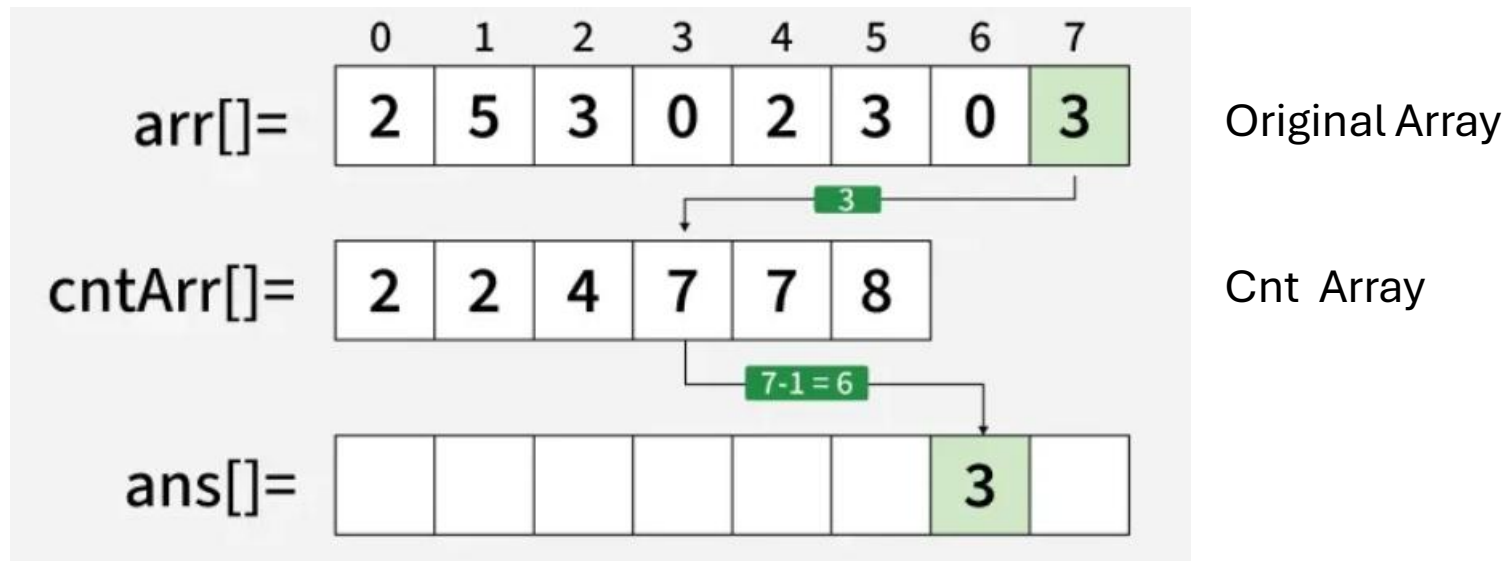


- **Step 4:** Store the cumulative sum or prefix sum of the elements of the `cntArr[]` by doing `cntArr[i-1]+cntArr[i]`

	0	1	2	3	4	5
cntArr[]=	2	2	4	7	7	8

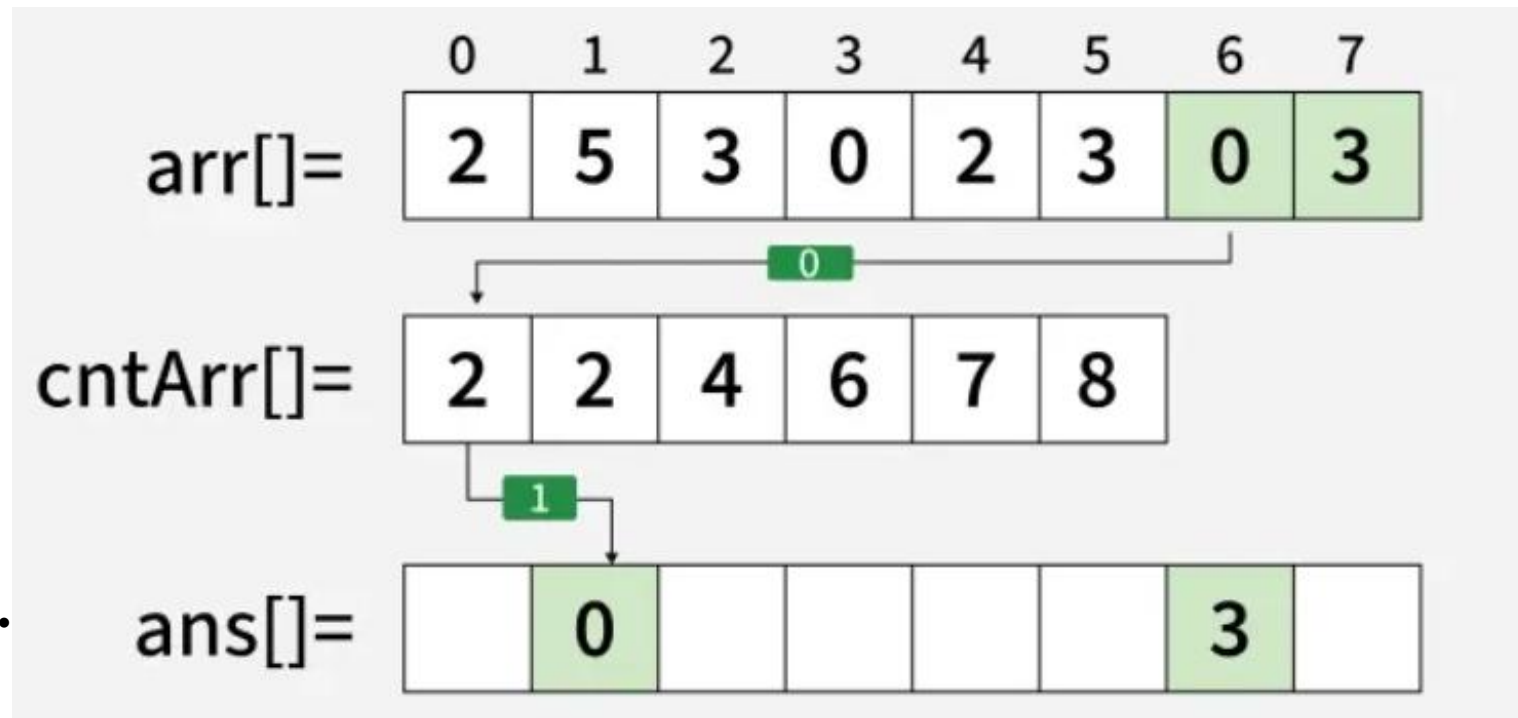
Working of Counting Sort cont....

- **Step 5:** Update $\text{ans}[\text{cntArr}[\text{arr}[i]-1]] = \text{arr}[i]$ and decrement $\text{cntArr}[\text{arr}[i]]$.
- Traverse the input array in reverse to maintain the order of equal elements, ensuring the sort remains stable.



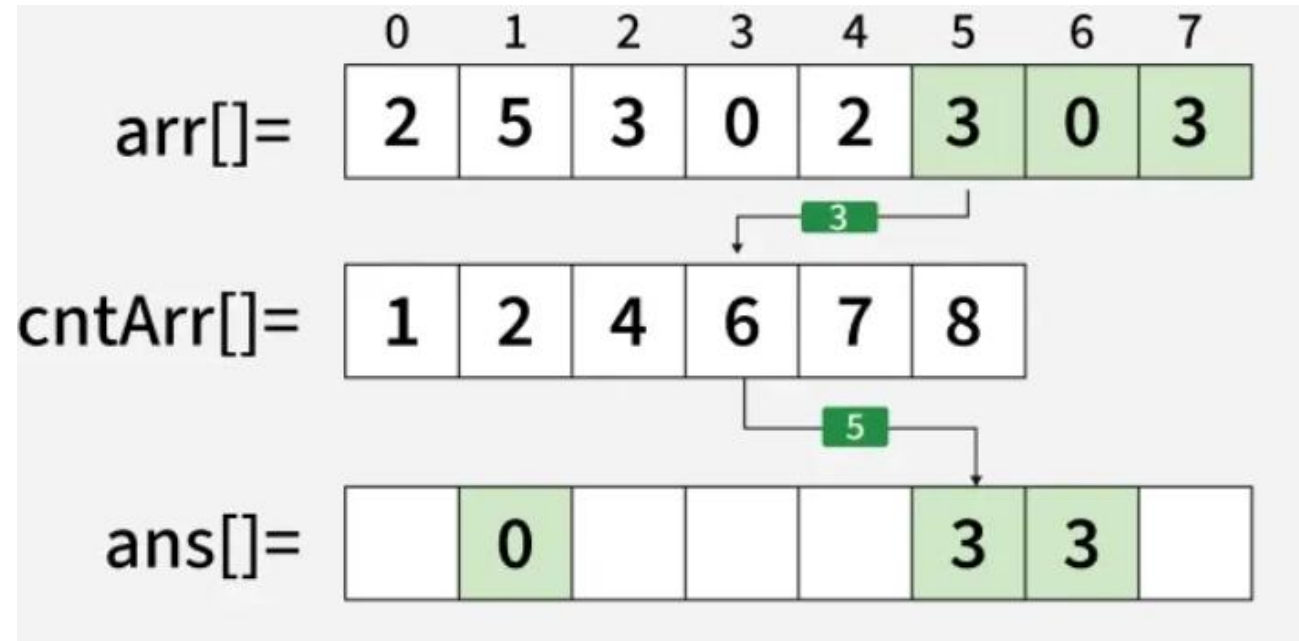
Working of Counting Sort cont....

- Step 6: update $\text{ans}[\text{cntArr}[\text{arr}[6]]-1] = \text{arr}[6]$
- Also, update $\text{cntArr}[\text{arr}[6]] = \text{cntArr}[\text{arr}[6]] + 1$



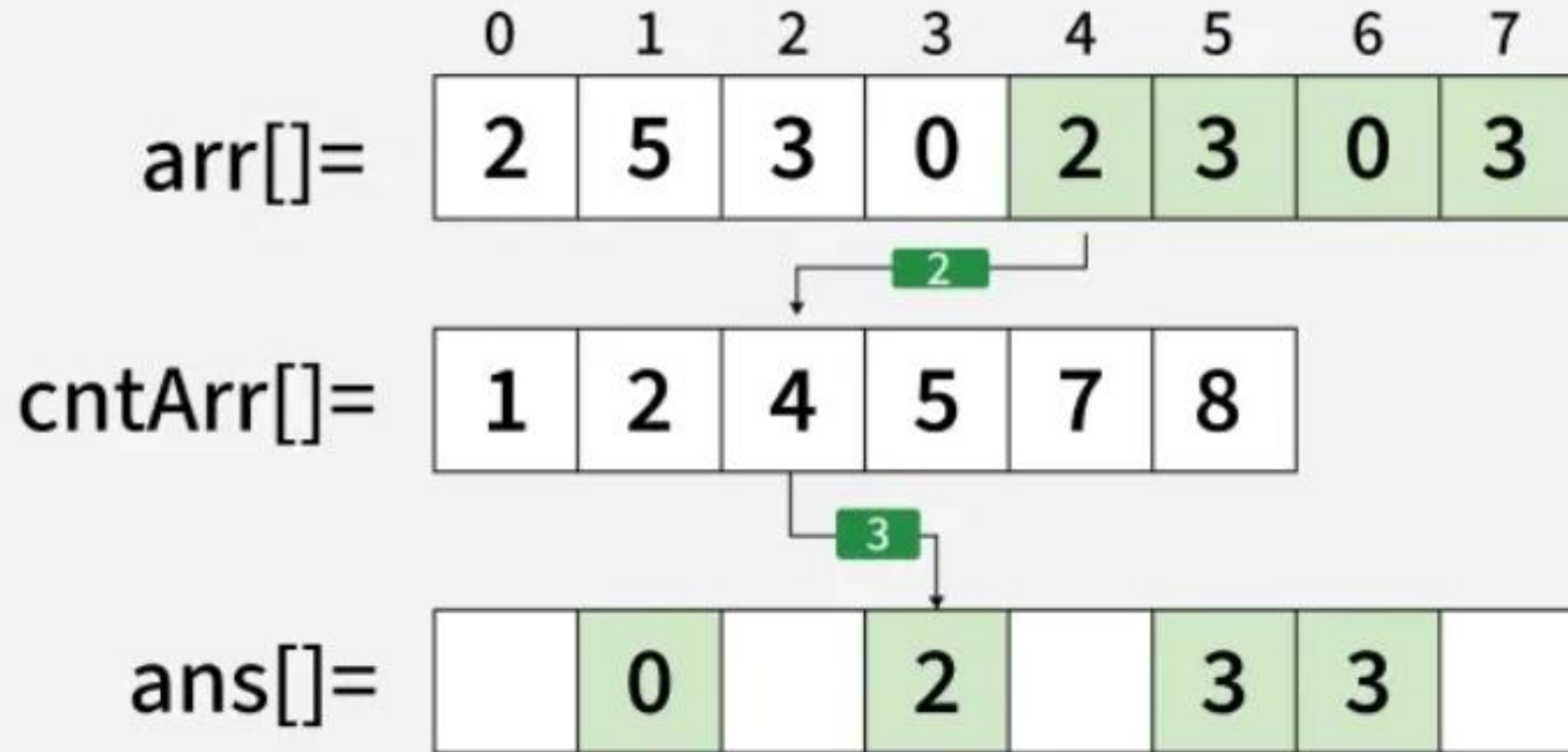
Working of Counting Sort cont....

- Step 7: Update $\text{ans}[\text{cntArr}[5]-1] = \text{arr}[5]$
- Also update $\text{cntArr}[\text{arr}[5]] = \text{cntArr}[\text{arr}[5]] + 1$...



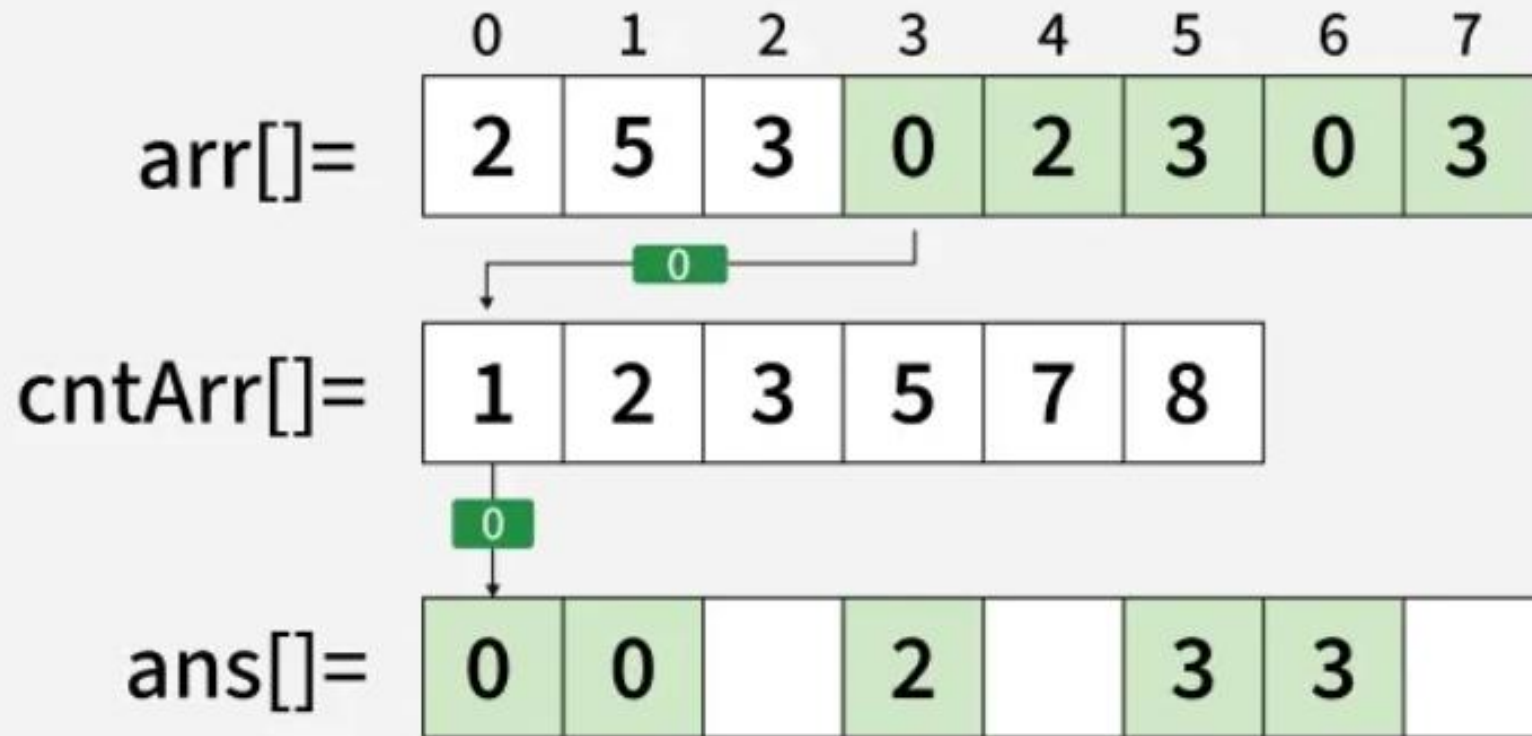
Working of Counting Sort cont....

Step 8 : Update $\text{ans}[\text{cntArr}[\text{arr}[4]] - 1] = \text{arr}[4]$
Also, update $\text{cntArr}[\text{arr}[4]] = \text{cntArr}[\text{arr}[4]] - 1$



Working of Counting Sort cont....

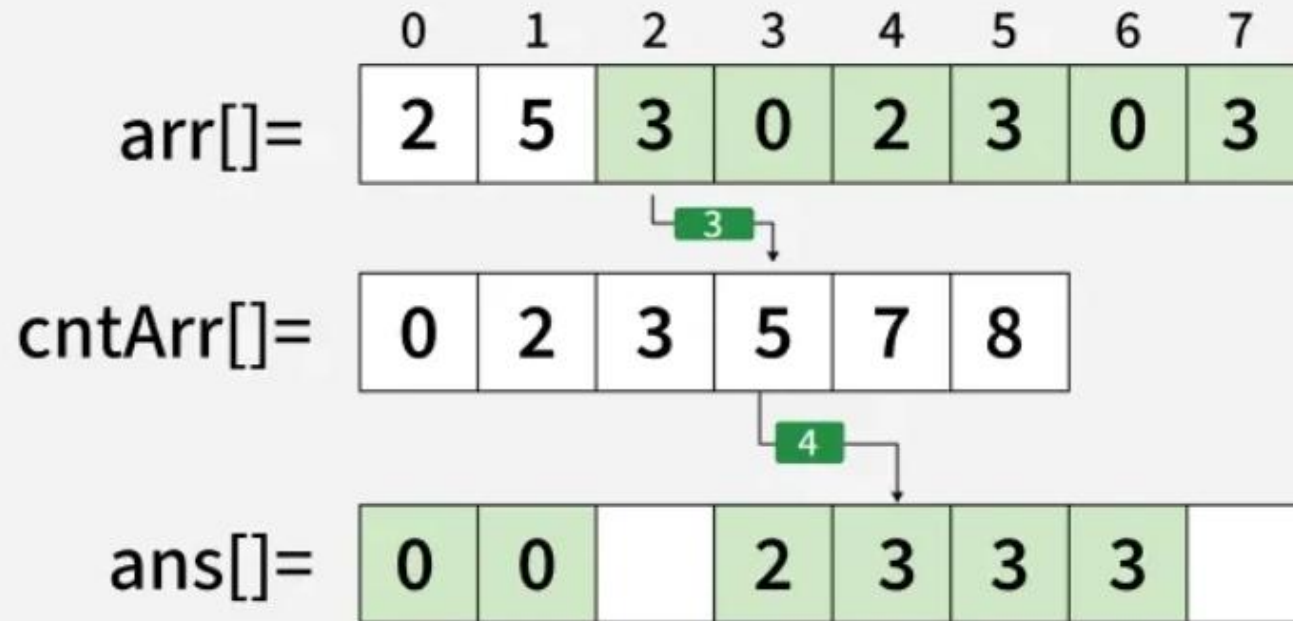
Step 9: Update $\text{ans}[\text{cntArr}[\text{arr}[3]] - 1] = \text{arr}[3]$
Also, update $\text{cntArr}[\text{arr}[3]] = \text{cntArr}[\text{arr}[3]] - 1$



Working of Counting Sort cont....

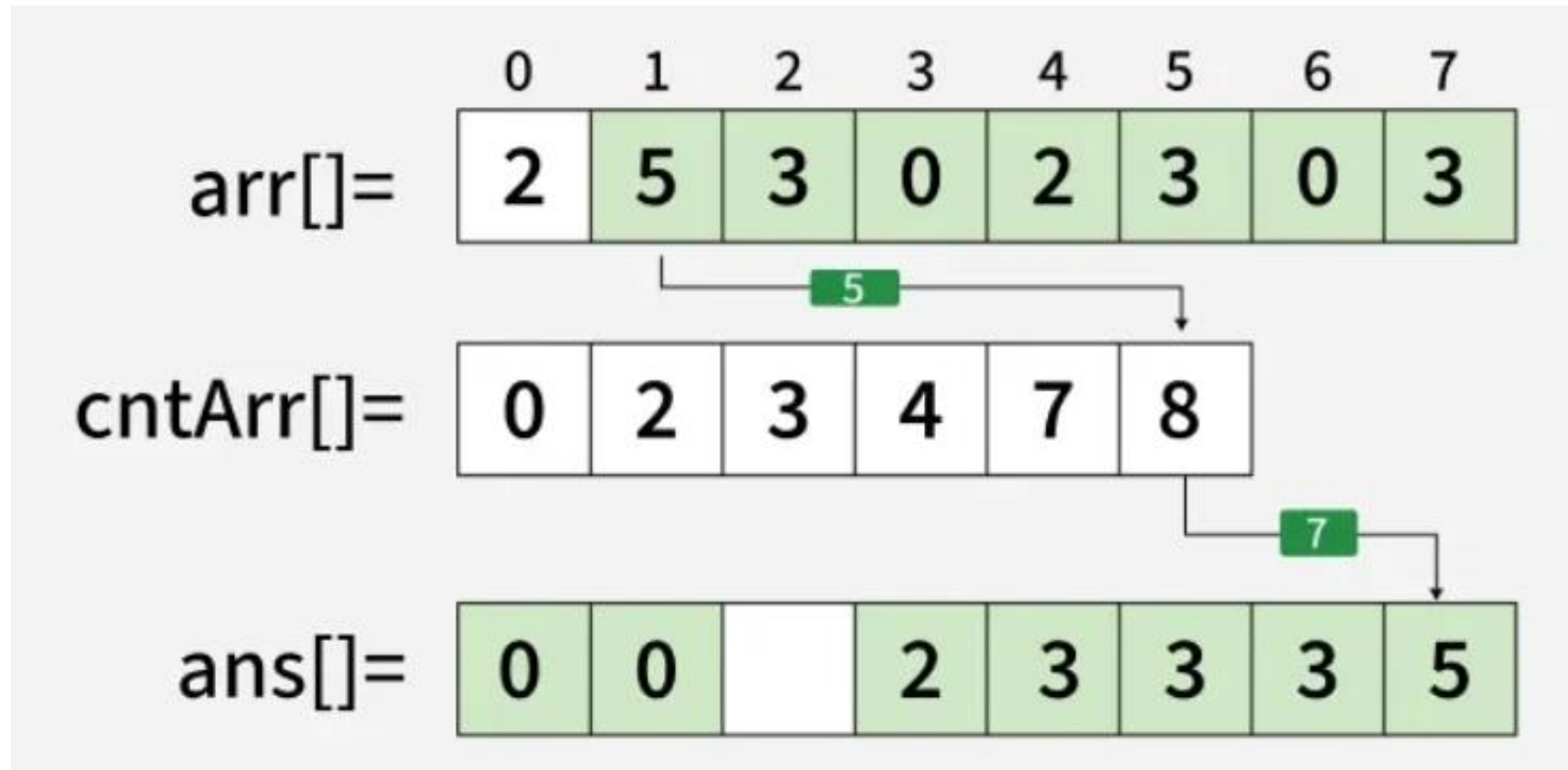
Step 10

Update $\text{ans}[\text{cntArr}[\text{arr}[2]] - 1] = \text{arr}[2]$
Also, update $\text{cntArr}[\text{arr}[2]] = \text{cntArr}[\text{arr}[2]] - 1$



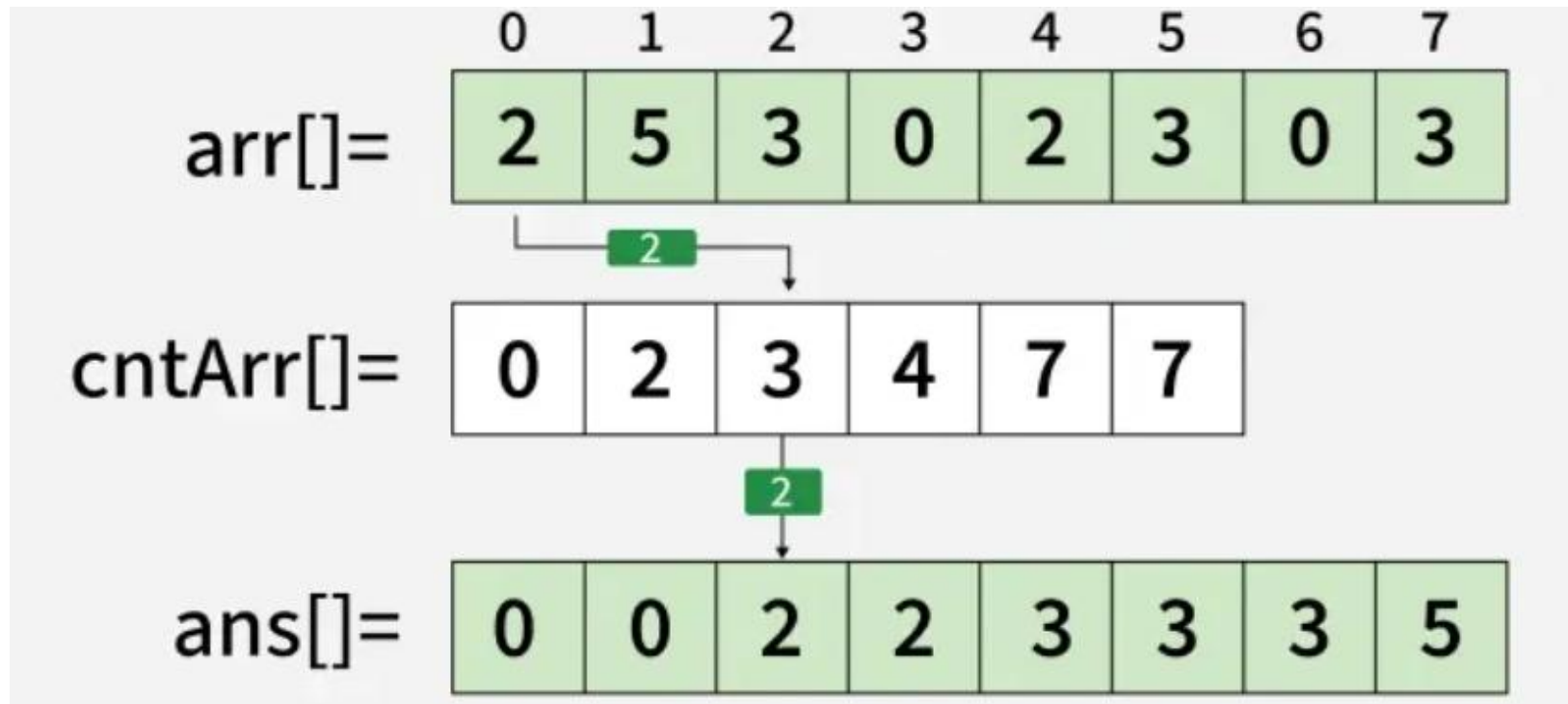
Working of Counting Sort cont....

- Step 11: Update $\text{ans}[\text{cntArr}[\text{arr}[1]] - 1] = \text{arr}[1]$
Also, update $\text{cntArr}[\text{arr}[1]] = \text{cntArr}[\text{arr}[1]] - 1$



Working of Counting Sort cont....

- Step 12: Update $\text{ans}[\text{cntArr}[\text{arr}[0]] - 1] = \text{arr}[0]$
Also, update $\text{cntArr}[\text{arr}[0]] = \text{cntArr}[\text{arr}[0]] - 1$



Counting Sort Algorithm

- Step 1:** Declare an auxiliary array **countArray[]** of size **max(inputArray[])+1** and initialize it with **0**.
- Step 2:** Traverse array **inputArray[]** and map each element of **inputArray[]** as an index of **countArray[]** array, i.e., execute **countArray[inputArray[i]]++** for **0 ≤ i < N**.
- Step 3:** Calculate the prefix sum at every index of array **inputArray[]**.
- Step 4 :** Create an array **outputArray[]** of size **N**.
- Step 5:** Traverse array **inputArray[]** from end and update **outputArray[countArray[inputArray[i]] - 1] = inputArray[i]**. Also, update **countArray[inputArray[i]] = countArray[inputArray[i]] - -**.

C code - Counting Sort Algorithm

```
#include <stdio.h>
#include <stdlib.h>
void countSort(int inputArray[], int N) {
    // Finding the maximum element of
    // array inputArray[]
    int M = 0;
    for (int i = 0; i < N; i++)
        if (inputArray[i] > M)
            M = inputArray[i];
```

C code - Counting Sort Algorithm (Cont....)

```
// Initializing countArray[] with 0
int* countArray = (int*)calloc(M + 1, sizeof(int));
    // Mapping each element of inputArray[]
    // as an index of countArray[] array
for (int i = 0; i < N; i++)
    countArray[inputArray[i]]++;
    // Calculating prefix sum at every index
    // of array countArray[]
for (int i = 1; i <= M; i++)
    countArray[i] += countArray[i - 1];
```

C code - Counting Sort Algorithm (Cont....)

```
// Creating outputArray[] from countArray[] array
int* outputArray = (int*)malloc(N * sizeof(int));
for (int i = N - 1; i >= 0; i--) {
    outputArray[countArray[inputArray[i]] - 1] = inputArray[i];
    countArray[inputArray[i]]--;
}
// Copying sorted elements back to inputArray[]
for (int i = 0; i < N; i++)
    inputArray[i] = outputArray[i];
```


C code - Counting Sort Algorithm (Cont...)

```
// Freeing dynamically allocated memory
    free(countArray);
    free(outputArray);
}
// Driver code
int main() {
    // Input array
    int inputArray[] = {2,5,3,0,2,3,0,3};
    int N = sizeof(inputArray) / sizeof(inputArray[0]);
```

C code - Counting Sort Algorithm (Cont...)

```
// Sorting the array
countSort(inputArray, N);
    // Printing the sorted array
    for (int i = 0; i < N; i++)
        printf("%d ", inputArray[i]);
    return 0;
}
```

Complexity Analysis of Counting Sort:

- **Time Complexity:** $O(N+M)$, where **N** and **M** are the size of **inputArray[]** and **countArray[]** respectively.
 - Worst-case: $O(N+M)$.
 - Average-case: $O(N+M)$.
 - Best-case: $O(N+M)$.
- **Auxiliary Space:** $O(N+M)$, where **N** and **M** are the space taken by **outputArray[]** and **countArray[]** respectively.

Bucket Sort Algorithm

- Bucket Sort is a sorting algorithm that divides the unsorted array elements into several groups called buckets.
- Each bucket is then sorted by using any of the suitable sorting algorithms or recursively applying the same bucket algorithm.
- Finally, the sorted buckets are combined to form a final sorted array.

Bucket Sort Algorithm:

- Create n empty buckets (Or lists) and do the following for every array element $arr[i]$.
 - Insert $arr[i]$ into $bucket[n*array[i]]$
 - Sort individual buckets using insertion sort.
 - Concatenate all sorted buckets.

Question 4

- Sort the following numbers using the Bucket sort algorithm
- Numbers are :**0.78, 0.17, 0.39, 0.26, 0.72, 0.94, 0.21, 0.12, 0.23, 0.68**
- Find the Worst-Case Time Complexity of the bucket sort

Radix Sort Algorithm

- Radix sort is the linear sorting algorithm that is used for integers.
- In Radix sort, there is digit by digit sorting is performed that is started from the least significant digit to the most significant digit.

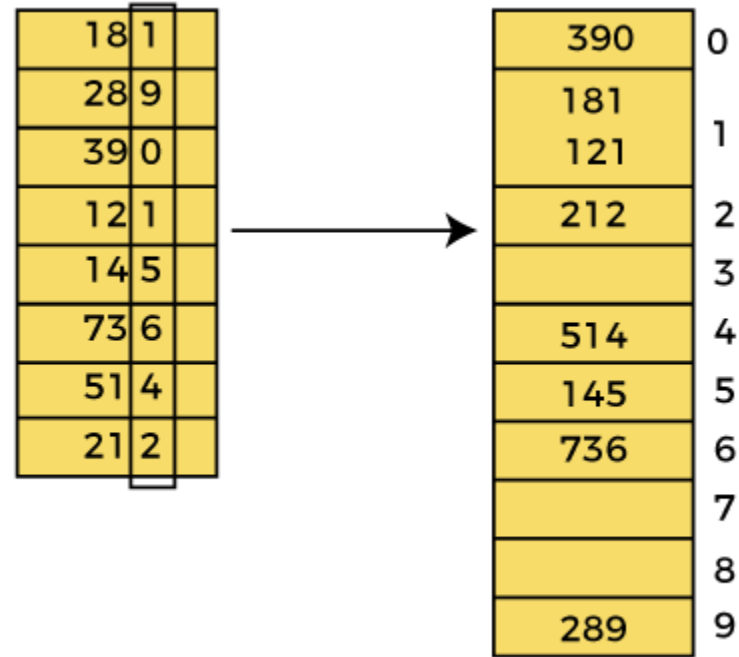
Working of Radix sort Algorithm

- Now let's see the working of radix sort in detail by using an example.
- To understand it more clearly, let's take an unsorted array and try to sort it using radix sort.
- It will make the explanation clearer and easier.

Working of Radix sort Algorithm

- Now, first sort the elements on the basis of unit place digits (i.e., $x = 0$).
- Here, we are using the counting sort algorithm to sort the elements.

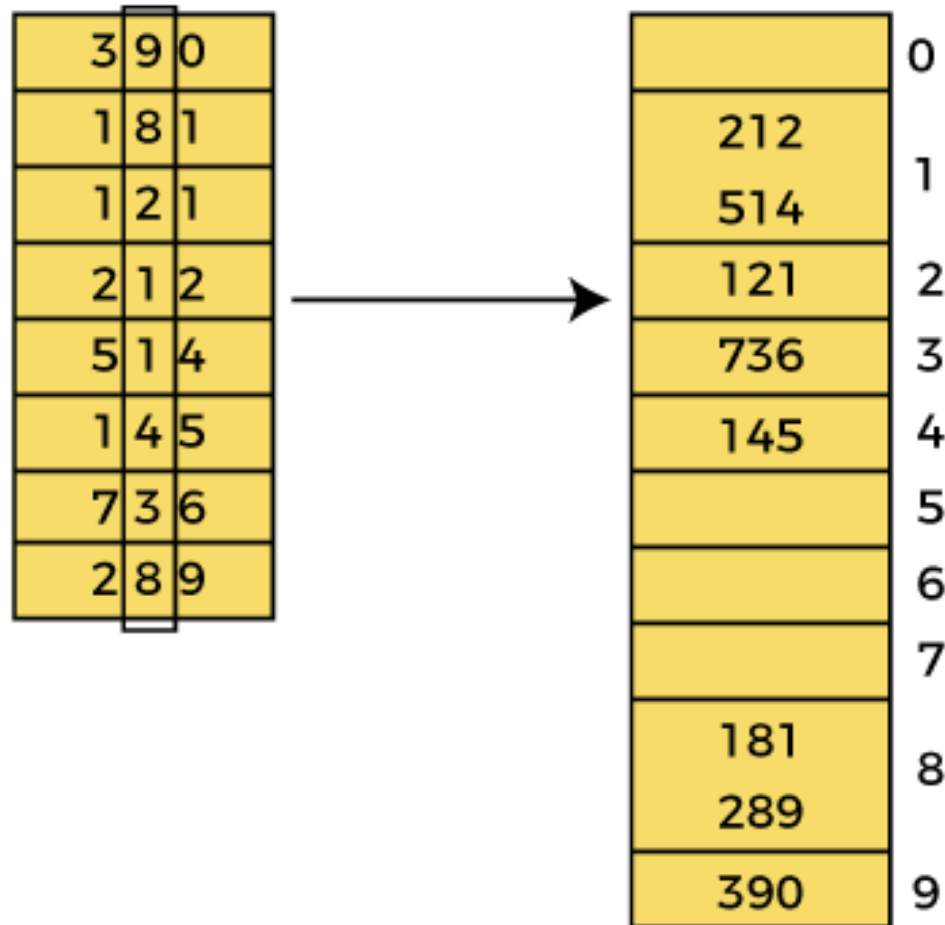
Pass 1: In the first pass, the list is sorted on the basis of the digits at 0's place.



After the first pass, the array elements are -

390	181	121	212	514	145	736	289
-----	-----	-----	-----	-----	-----	-----	-----

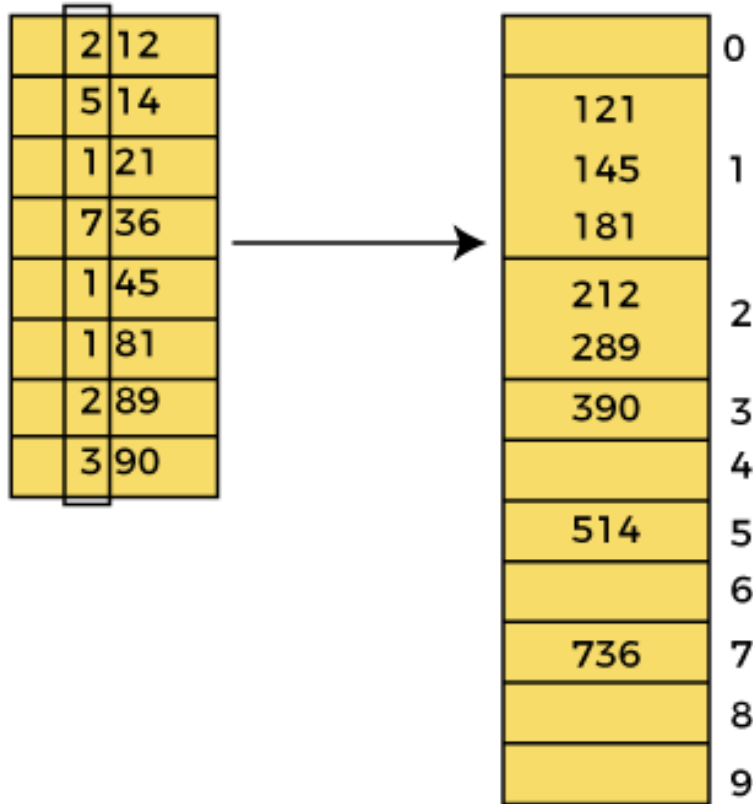
Pass 2: in this pass, the list is sorted on the basis of the next significant digits (i.e., digits at 10th place)



After the second pass, the array elements are -

212	514	121	736	145	181	289	390
-----	-----	-----	-----	-----	-----	-----	-----

Pass 3: In this pass, the list is sorted on the basis of the next significant digits (i.e. digits at 100th place).



After the third pass, the array elements are -

121	145	181	212	289	390	514	736
-----	-----	-----	-----	-----	-----	-----	-----

Now, the array is sorted in ascending order.

Radix sort Algorithm

- Steps Involved in Radix sort to sort an array in ascending order are
 - - Find the maximum element of the array, let it be max
 - Find the number of digits in max, let it be k .
 - For each, i^{th} ranging from 1 To k , apply the counting sort algorithm for the i^{th} least-significant digit of each element. If any element has less than i digits
 - Homework: coding , test run????

Question 6

- Explain the differences between the **Bucket Sort** algorithm and the **Radix Sort** algorithm.